Experiment No. 9

Object Detection and Recognition using Deep Learning Models on Standard Image Datasets

Name = Harsh Kaushik

CU_ID = CU24250248

Subject = Computer Vision Lab

Branch = B.Tech (CSE)

Aim: To implement a deep learning-based object detection and recognition system using a pre-trained YOLOv8 model and evaluate its performance on standard images for identifying and localizing multiple objects.

Purpose: Object detection combines object localization and classification. Unlike image classification, it identifies multiple objects and provides bounding boxes, class labels, and confidence scores.

Prerequisites: Basic Digital Image Processing, Python, OpenCV, Ultralytics YOLO, NumPy, Matplotlib, CNN basics, and Google Colab/Jupyter.

```
!pip -q install ultralytics opencv-python-headless matplotlib pandas
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.7/46.7 kB 1.4 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.5/1.5 MB 14.4 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 53.2/53.2 kB 2.7 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 88.7/88.7 kB 3.3 MB/s eta 0:00:00
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import time
from google.colab import files
from ultralytics import YOLO

print("Libraries imported successfully.")
```

```
Creating new Ultralytics Settings v0.0.8 file ✅
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics
Libraries imported successfully.
```

Step 1: Load a Pre-trained YOLOv8 Model

YOLOv8 Nano (`yolov8n.pt`) is used because it is lightweight and suitable for Google Colab. The model is pre-trained on the MS COCO dataset.

```
model = YOLO("yolov8n.pt")
print("YOLOv8 model loaded successfully.")
print("Number of supported classes:", len(model.names))
```

```
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt to 'yolov8n.pt': 100% ━━━━━━━━━━━━ 6.2MB 4
YOLOv8 model loaded successfully.
Number of supported classes: 80
```

Step 2: Upload a Test Image

Upload an image containing multiple everyday objects. The COCO-trained model can recognize common classes such as person, car, bicycle, bus, dog, cat, chair, bottle, and more.

```
uploaded = files.upload()
file_name = next(iter(uploaded))
image = cv2.imread(file_name)

if image is None:
```

```
    raise ValueError("Image could not be loaded. Please upload a valid image.")

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
print("Image:", file_name)
print("Image shape:", image.shape)

plt.figure(figsize=(10, 6))
plt.imshow(image_rgb)
plt.title("Original Input Image")
plt.axis("off")
plt.show()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving images (2).jpg to images (2).jpg
Image: images (2).jpg
Image shape: (224, 400, 3)
```

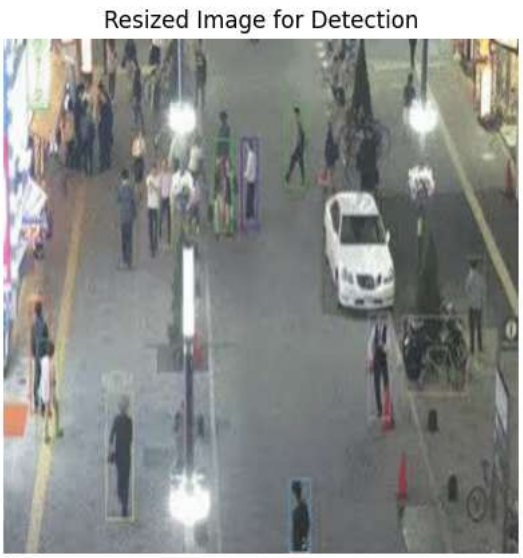


Step 3: Image Preprocessing

YOLO performs the required resizing and tensor preprocessing internally. This cell shows a 640×640 resized version for inspection.

```
preprocessed = cv2.resize(image_rgb, (640, 640))

plt.figure(figsize=(8, 5))
plt.imshow(preprocessed)
plt.title("Resized Image for Detection")
plt.axis("off")
plt.show()
```



Step 4: Perform Object Detection

The model predicts bounding boxes, class IDs, and confidence scores. A confidence threshold of 0.25 is used.

```
start_time = time.time()
results = model.predict(source=image_rgb, imgsz=640, conf=0.25, verbose=False)
inference_time = time.time() - start_time

result = results[0]
print(f"Inference time: {inference_time:.4f} seconds")
print(f"Approximate FPS: {1 / inference_time:.2f}")
```

```
Inference time: 8.2777 seconds
Approximate FPS: 0.12
```

Step 5: Display Bounding Boxes, Class Labels, and Confidence Scores

```
annotated = result.plot()
annotated_rgb = cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(12, 8))
plt.imshow(annotated_rgb)
plt.title("YOLOv8 Object Detection Result")
plt.axis("off")
plt.show()
```



Step 6: Extract Detection Information

The table contains the detected class, confidence score, and bounding-box coordinates `(x1, y1, x2, y2)`.

```
detections = []

if result.boxes is not None and len(result.boxes) > 0:
    boxes = result.boxes.xyxy.cpu().numpy()
    confidences = result.boxes.conf.cpu().numpy()
    class_ids = result.boxes.cls.cpu().numpy().astype(int)

    for box, confidence, class_id in zip(boxes, confidences, class_ids):
        x1, y1, x2, y2 = box
        detections.append({
            "Class": model.names[class_id],
            "Confidence": round(float(confidence), 4),
            "x1": round(float(x1), 1),
            "y1": round(float(y1), 1),
            "x2": round(float(x2), 1),
            "y2": round(float(y2), 1)
        })

detection_df = pd.DataFrame(detections)

if len(detection_df) > 0:
    display(detection_df)
else:
    print("No objects were detected above the confidence threshold.")
```

	Class	Confidence	x1	y1	x2	y2
0	car	0.7198	248.5	65.6	304.7	118.5
1	person	0.5002	286.0	121.5	303.0	169.0
2	person	0.4484	21.4	113.7	35.3	162.4
3	person	0.3487	89.0	55.9	103.0	99.0
4	person	0.2563	82.5	158.7	99.8	208.0

Step 7: Detection Summary

```
if len(detection_df) > 0:
    print("Total detected objects:", len(detection_df))
    print("Average confidence:", round(detection_df["Confidence"].mean(), 4))
    print("\nDetected object categories:")
    print(detection_df["Class"].value_counts())
else:
    print("No detections available for summary.")
```

```
Total detected objects: 5
Average confidence: 0.4547

Detected object categories:
Class
person    4
car       1
Name: count, dtype: int64
```

Step 8: Evaluate Detection Results

Correct detections: Objects whose class and location are correctly identified.

False detections: Predictions that do not correspond to a real target object or have an incorrect class.

Missed detections: Real objects that the model fails to detect.

A quantitative precision, recall, mAP, or IoU evaluation requires ground-truth annotations. The uploaded image is therefore evaluated qualitatively unless annotation files are also supplied.

```
confidence_thresholds = [0.25, 0.50, 0.75]
threshold_summary = []

for threshold in confidence_thresholds:
    threshold_results = model.predict(
        source=image_rgb, imgsz=640, conf=threshold, verbose=False
    )
    boxes = threshold_results[0].boxes
    count = len(boxes) if boxes is not None else 0
    threshold_summary.append({
        "Confidence Threshold": threshold,
        "Detected Objects": count
    })

threshold_df = pd.DataFrame(threshold_summary)
display(threshold_df)
```

	Confidence Threshold	Detected Objects
0	0.25	5
1	0.50	2
2	0.75	0

Step 9: Model Analysis

Parameter	Observation
Model	YOLOv8 Nano
Training dataset	MS COCO
Detection type	One-stage object detection
Output	Bounding boxes, class labels, confidence scores
Speed	Designed for fast inference
Advantage	Good balance of speed and accuracy
Limitation	Small, crowded, blurred, or occluded objects can be difficult to detect

Step 10: Advantages and Limitations

Advantages

- Pre-trained models reduce the need to train from scratch.
- YOLO performs localization and classification in one detection pipeline.

- Bounding boxes and confidence scores are produced directly.
- Suitable for real-time applications with suitable hardware.
- Transfer learning makes modern object detection practical.

Limitations

- Performance depends on training data and supported classes.
- Small, overlapping, blurred, or heavily occluded objects are difficult.
- Higher accuracy models generally require more computation.
- COCO-trained models may not recognize domain-specific objects without fine-tuning.

Questions and Answers

1. Differentiate between image classification, object detection, and image segmentation.

Image classification assigns a label to an image or region. **Object detection** identifies object classes and their locations using bounding boxes. **Image segmentation** assigns labels to individual pixels, giving more precise object boundaries.

2. Explain the working principle of YOLO.

YOLO treats detection as a single prediction problem. The image passes through a neural network that predicts bounding boxes, objectness/confidence values, and class probabilities. Non-Maximum Suppression removes highly overlapping duplicate predictions.

3. Compare YOLO, SSD, and Faster R-CNN.

Model	General Speed	Accuracy Characteristics	Typical Applications
YOLO	High	Strong speed-accuracy balance	Real-time detection, robotics, surveillance
SSD	High	Good speed with moderate accuracy	Embedded and real-time systems
Faster R-CNN	Lower	Often strong localization accuracy	Research and accuracy-focused applications

Exact performance depends on model version, hardware, image size, and dataset.

4. What is a bounding box?

A bounding box is a rectangle defined by coordinates around a detected object. It specifies where the object is located.

5. Explain confidence score and IoU.

A **confidence score** indicates how strongly the model supports a detection. **Intersection over Union (IoU)** measures overlap between predicted and ground-truth boxes: `IoU = Area of Intersection / Area of Union`. IoU is widely used to evaluate localization quality.

6. Advantages of pre-trained deep learning models over traditional techniques.

Pre-trained models learn complex visual features from large datasets and can handle variation in pose, scale, and background. Traditional methods often depend on manually designed features and tuned rules. Pre-trained models also support transfer learning.

7. Why are MS COCO and Pascal VOC widely used?

They provide standardized annotated images, object classes, bounding boxes, and established evaluation protocols, making training and comparison of object detectors easier.

8. Five real-world applications.

1. Autonomous driving
2. Video surveillance and security
3. Retail analytics and inventory monitoring
4. Medical image analysis
5. Industrial inspection and robotics

9. Common challenges in complex environments.

Poor lighting, shadows, cluttered backgrounds, scale variation, occlusion, overlapping objects, motion blur, low resolution, unusual viewpoints, and visually similar classes.